#3 We are at the target node for the target key: the tree stores it only if current node is a key node.
#4 Since `s` is not the empty string, we can see if there is an edge matching it, even partially.
#5 If there is an edge with a common prefix with `s`, and actually the edge's label is a prefix to `s`, then we recursively search the remaining characters of (stored in `sSuffix`) into the subtree linked by the edge. This is case 1 of the 4 possible matches in figure 6.15.
#6 Otherwise, we are in one of cases #2 to #4: the key is certainly not stored in the tree, and we can return.

This method just looks for an edge with a common prefix with the target string `s`, if any: remember that all edges can't have any prefix in common, so there can be at most one starting with same character as `s`.

It returns some useful information that the caller can use to decide the action to take: the longest common prefix between searched string and edge's label, and the suffixes of these two strings (with respect to their common prefix).

At line #5 of listing 6.13, we use this information to distinguish between the four match cases illustrated in figure 6.15: the only positive case for search is the first one, so we need to check that there is an edge whose label is a prefix of `s`.

Listing 6.14 method matchEdge for Radix Tries

```
function matchEdge(node, s)
                                                    #1
```